SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Name: S Ilamparithi **Register Number:** 1916260093

Code of Conduct:

*I **S Ilamparithi / 1916260093** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

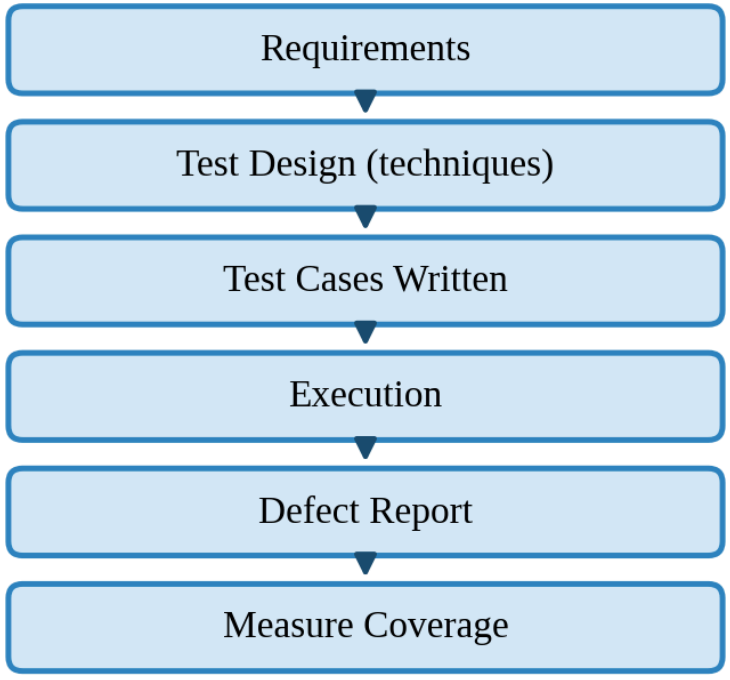
Signature of the student: *S Ilamparithi*

Question 1

A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer

- **Engineering approach:** Testing is planned, designed, executed and measured, not random clicking.
- **Test design techniques:** Equivalence partitioning, boundary value analysis and decision tables are applied to every requirement.
- **Documented test cases:** Each has an ID, input, steps and expected result, so any tester gets the same result.
- **Traceability:** Every test case maps to a requirement, so no feature is left untested.
- **Example:** For a login page, random testing tries 2-3 passwords. Structured design covers valid, invalid, empty, maximum length and special characters, which finds the missed defects.
- **Metrics:** Coverage and defect detection rate are measured, so improvement can be proved.
- **Outcome:** Testing becomes predictable, repeatable and reviewable.

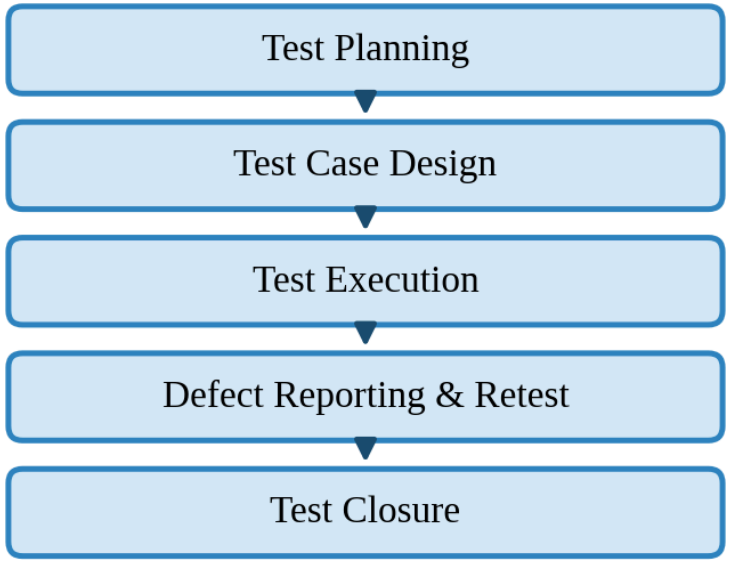


Question 2

A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer

- **Defined stages:** Planning, design, execution, defect reporting and closure, each with entry and exit criteria.
- **Repeatable:** Every release is tested the same way, so results do not depend on who tests.
- **Example 1:** Fund transfer is tested for insufficient balance, timeout and double debit before release.
- **Example 2:** Interest calculation has fixed test data and expected results, so wrong amounts are caught.
- **Benefits:** Fewer production failures, audit trail for compliance, lower cost and higher customer trust.
- **Standards:** Follows IEEE 829 / ISTQB style documents such as test plan and test report.
- **Real-time impact:** Early detection avoids financial loss and downtime in banking.

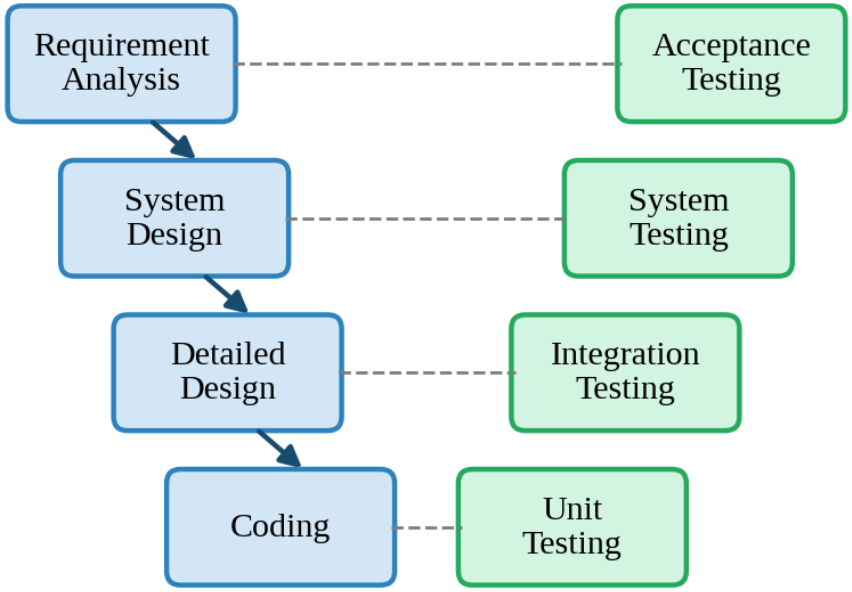


Question 3

During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer

- **Test with every phase:** Each development phase has a matching test level (V-model).
- **Requirements:** Acceptance test planning and requirement review.
- **Design:** System test and integration test planning, e.g. cart-to-payment interface.
- **Coding:** Unit testing by developers, then integration and system testing.
- **Discipline gives quality:** Defects are caught in the phase where they appear, so fewer integration and system defects reach the customer.
- **Entry/exit criteria:** A phase ends only when its test exit criteria are met.
- **Impact:** Lower rework cost and predictable release quality.



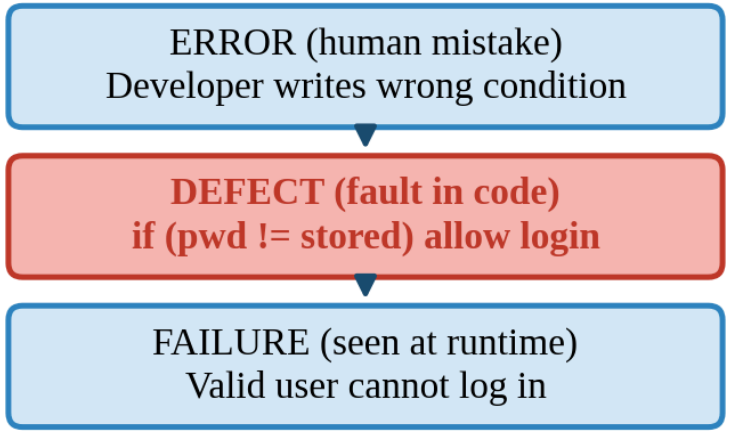
Each development phase has a matching test level

Question 4

A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer

- **Classification:** The mismatch seen during execution is a failure.
- **Error (mistake):** A human mistake by the developer, e.g. typing the wrong comparison.
- **Defect (bug/fault):** The wrong code that results from the error, e.g. password check uses != instead of ==.
- **Failure:** The defect executes and the system behaves wrongly, e.g. a valid user is rejected.
- **Relationship:** Error leads to defect, and defect leads to failure. Not every defect causes a failure.
- **Tip:** Testers report failures. Developers find the defect behind them.
- **Fix:** Correct the condition to ==, then retest and run regression.

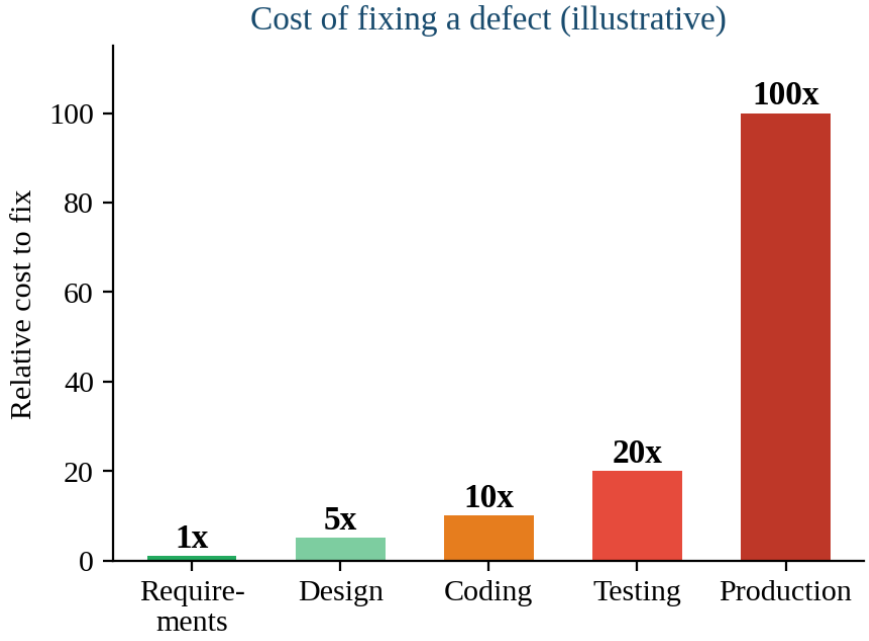


Question 5

A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer

- **Principle:** Early testing. Testing starts in the requirement phase, not after coding.
- **Why:** The cost of fixing a defect grows sharply in each later phase.
- **How applied:** Review requirements, design and code, and prepare test cases while development is going on.
- **Example:** A wrong tax rule found in a requirement review costs a few minutes to correct. The same defect found after release needs code change, retesting, redeployment and customer compensation.
- **Result:** Lower cost, shorter schedule and better quality.
- **Other principles:** Testing shows presence of defects, and exhaustive testing is impossible.
- **Practice:** Static testing (reviews, walkthroughs) finds defects without running code.

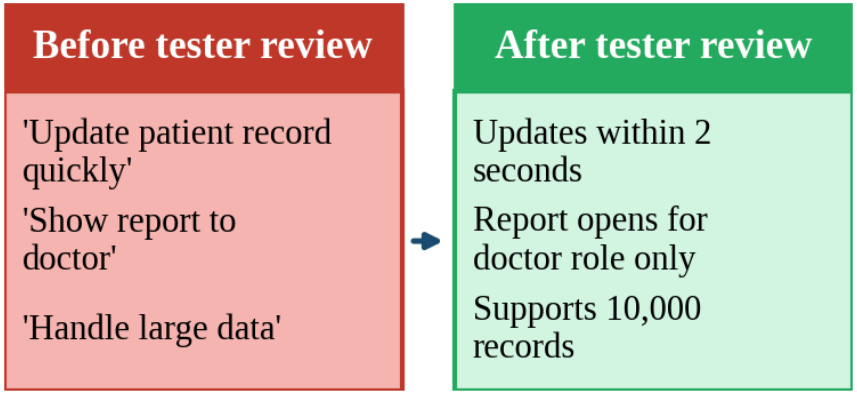


Question 6

During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer

- **Review requirements:** The tester checks them for being clear, complete, consistent and testable.
- **Raise questions:** Vague words like quickly, user-friendly or appropriate are queried and made measurable.
- **Define acceptance criteria:** Test conditions are written along with the requirement.
- **Example:** 'Show patient report quickly' becomes 'Report opens within 3 seconds for 1000 records'.
- **Importance:** Ambiguity is removed before design and coding, so defects are prevented rather than found late.
- **Traceability:** Each requirement is linked to test cases, so nothing is missed.
- **Benefit:** Healthcare errors can affect patient safety, so early clarity is critical.

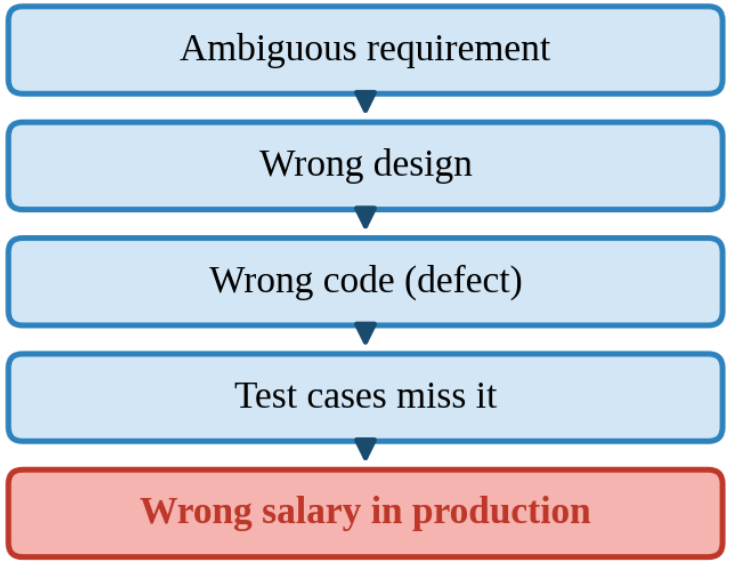


Question 7

A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer

- **Origin:** Most defects are injected early. An unclear requirement is misunderstood in design, coded wrongly and then missed in testing.
- **Example:** 'Calculate overtime pay' does not state the rate or the hours. The developer assumes 1.5x, but the policy is 2x on holidays, so salary is wrong.
- **Spread:** Design, code and test cases all inherit the same wrong assumption.
- **Prevention:** Clear, measurable, testable requirements with formulas and examples.
- **Also:** Requirement reviews and a traceability matrix.
- **Root causes:** Missing details, conflicting statements and unstated assumptions.
- **Check:** Use a requirement review checklist (clear, complete, consistent, testable).

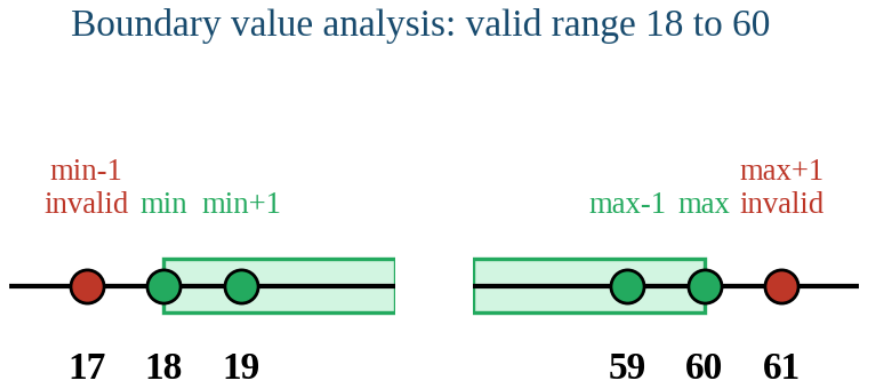


Question 8

A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.

Answer

- **Classification:** Functional (input validation / boundary) defect, high severity because the system crashes.
- **Justification:** The code does not handle the limit values correctly, a typical off-by-one error such as using < instead of <=.
- **Boundary value analysis:** Test at and around each limit: min-1, min, min+1, max-1, max, max+1.
- **Example:** Age field accepts 18 to 60. Tests: 17, 18, 19, 59, 60, 61.
- **Prevention:** The crash at 60 or 61 would have been found before release.
- **Type:** Also called a boundary value defect, found by black-box testing.
- **Related:** Equivalence partitioning reduces the number of valid and invalid cases to test.

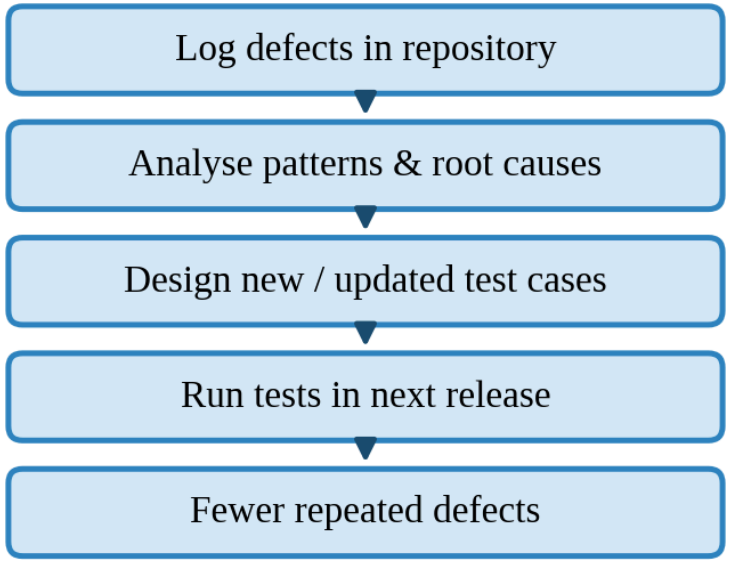


Question 9

A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer

- **Repository:** Stores defect ID, module, severity, steps to reproduce and root cause.
- **Analyse:** Find defect-prone modules and the most common defect types.
- **Design tests:** Add test cases for every past defect and for similar patterns, e.g. repeated date-format errors.
- **Reuse:** Past defects feed the regression suite.
- **Role in quality:** Prevents repeated defects, guides risk-based testing and provides metrics for improvement.
- **Metrics:** Defect density and defect age show where quality is weak.
- **Feedback:** Lessons learned are shared with developers and the whole team.

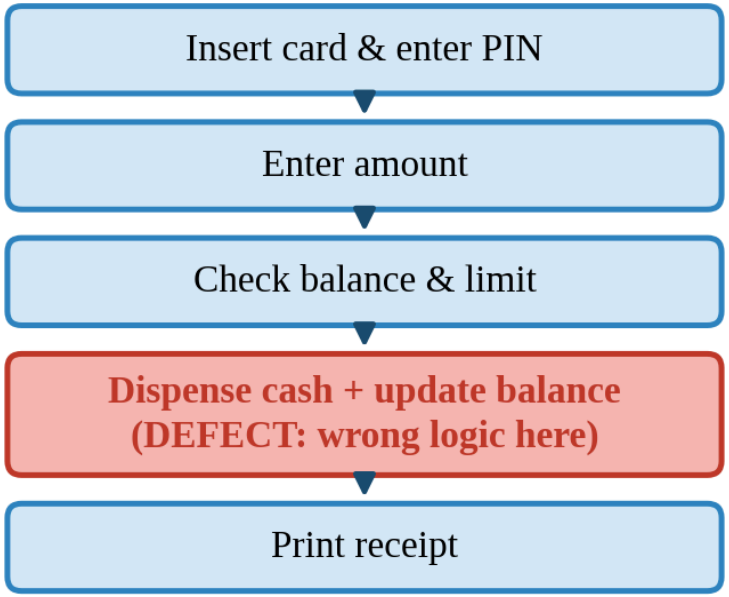


Question 10

An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer

- **Functional defect:** The system does not behave as the specification says.
- **Example:** User withdraws Rs. 5,000 from Rs. 20,000 balance. Cash is dispensed but the balance is not updated, or it is deducted twice.
- **Why missed:** Only normal cases were tested. Failure cases such as network error in the middle of a transaction were not.
- **Prevention:** Requirement review, decision table and state transition tests for each withdrawal path.
- **Also:** Code review and regression tests.
- **Severity:** Critical, since money is involved.
- **Negative tests:** Wrong PIN, card swallowed and cash tray jam must also be tested.

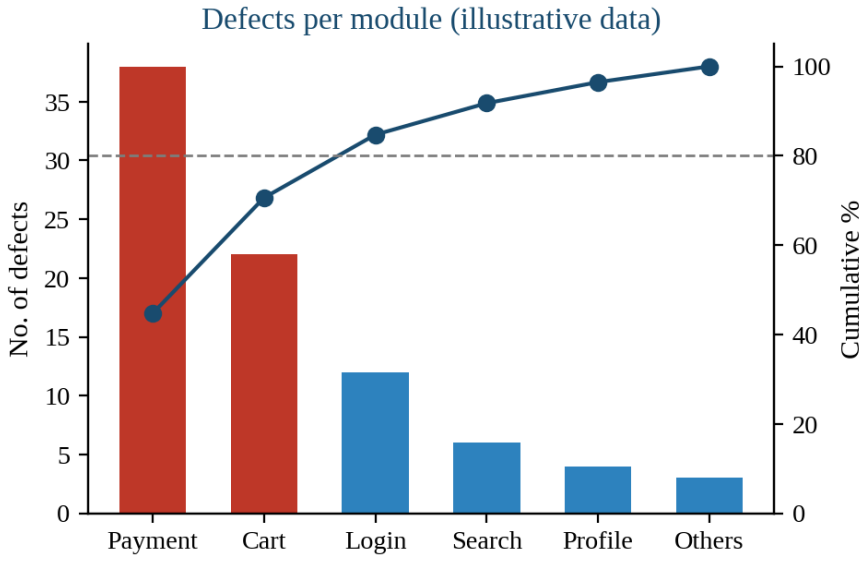


Question 11

A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.

Answer

- **Defect clustering:** A small number of modules contain most defects (Pareto rule, about 80% in 20%).
- **Priorities:** In the chart, Payment and Cart hold about 60% of defects, so they get testing first.
- **Action:** More test depth, extra reviews and more regression for those modules.
- **Efficiency:** Effort goes where the risk is, so more defects are found in less time.
- **Caution:** Other modules still get basic testing.
- **Related principle:** Pesticide paradox, so test cases must be reviewed and updated regularly.
- **Data source:** The defect repository and previous releases show the clusters.

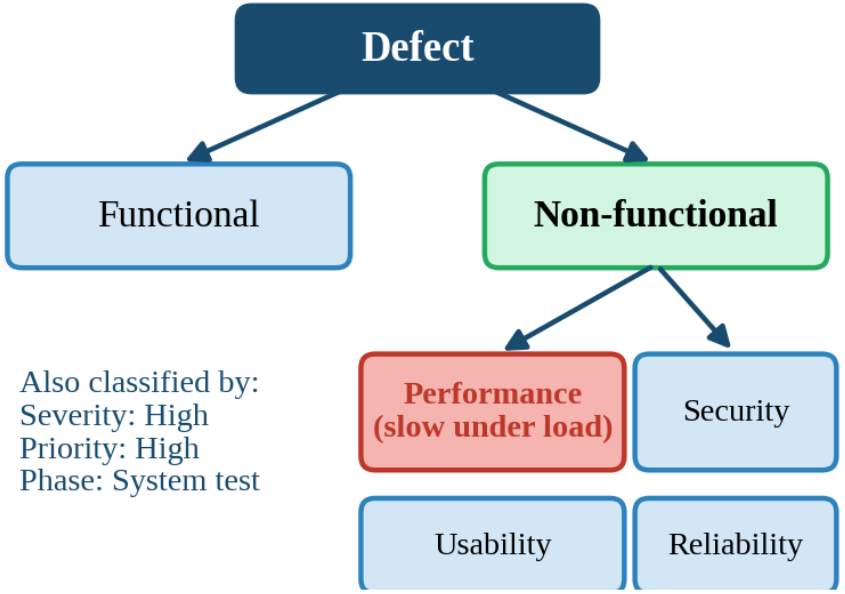


Question 12

A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.

Answer

- **Type:** Non-functional defect, specifically a performance defect (the features work but response time is poor).
- **Other attributes:** Severity High, priority High, found in system / load testing, origin in design or code.
- **Helps resolution:** It goes to the right team, e.g. performance engineers and DB team.
- **Prioritisation:** Severity and priority decide what is fixed first.
- **Analysis:** Classification shows trends such as most defects coming from design.
- **Tools:** JMeter or LoadRunner simulate heavy user load.
- **Fixes:** Caching, query tuning, load balancing and code optimisation.

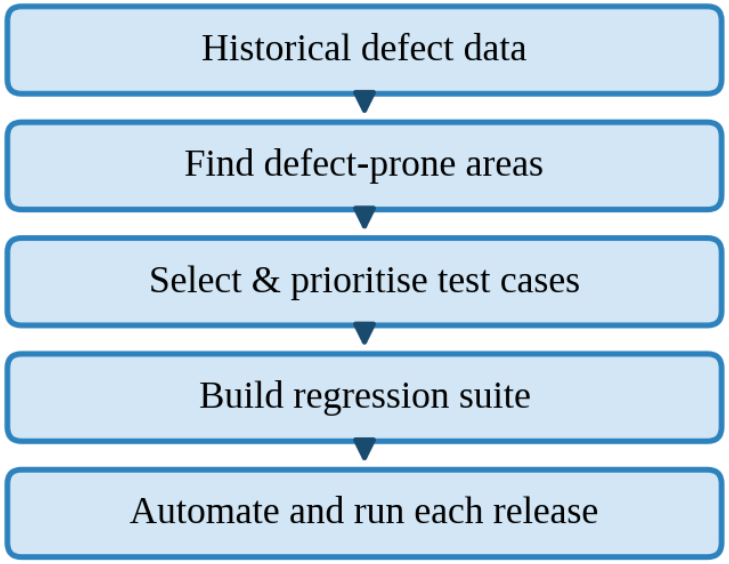


Question 13

A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer

- **Analyse history:** Find modules with repeated defects and high-severity defects.
- **Add tests:** Each fixed defect gets a regression test case, so it cannot come back unnoticed.
- **Prioritise:** Run high-risk and defect-prone areas first.
- **Remove waste:** Drop test cases that never find defects.
- **Result:** Better coverage of risky areas, less execution time. Stable tests can be automated.
- **Selection:** Retest all high-priority tests, and sample the rest.
- **Tools:** Selenium or similar tools run the suite automatically after every build.

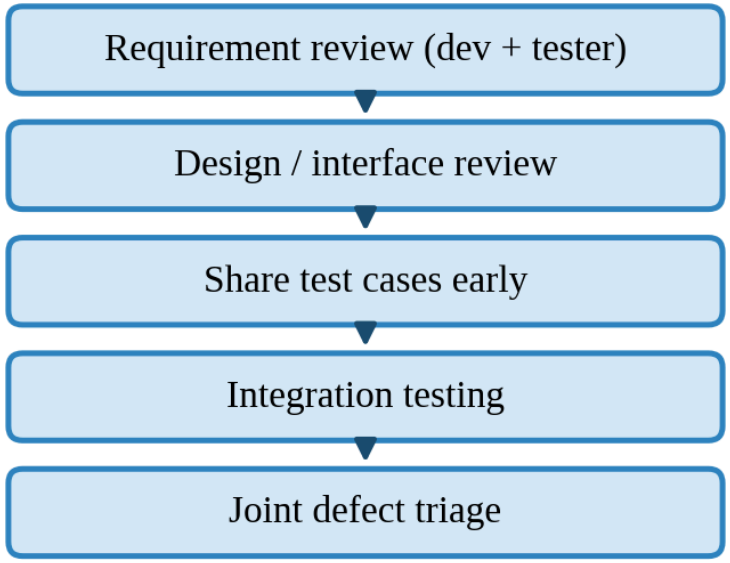


Question 14

During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer

- **Role of tester:** Reviews requirements and interface design, shares test cases with developers and clarifies doubts.
- **Integration focus:** Verifies data formats and API contracts between modules.
- **Communication:** Clear defect reports and regular triage meetings with developers.
- **Early involvement:** Interface mismatches are noticed at design stage, not during integration.
- **Result:** Fewer integration defects and fewer rejected or reopened defects.
- **Shared tools:** Common tracker and daily stand-up reduce misunderstanding.
- **Contract tests:** API and data format checks at each interface.

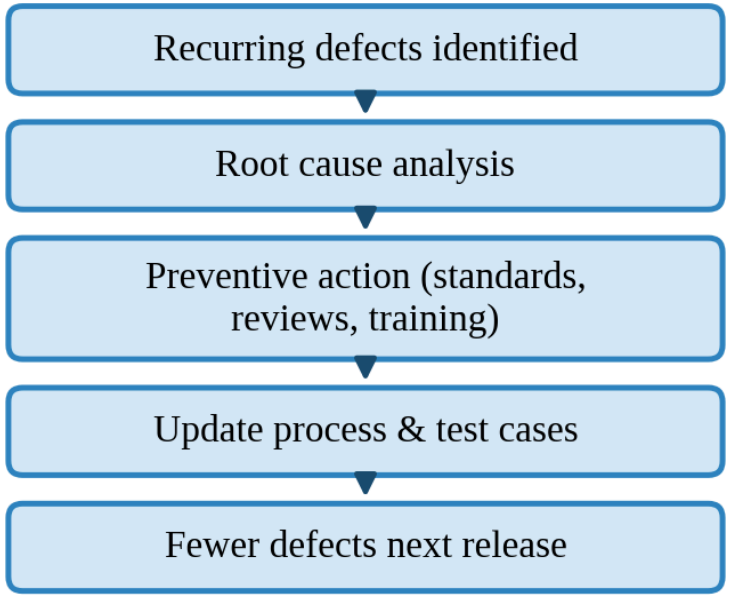


Question 15

A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.

Answer

- **Root cause analysis:** Use the 5-Whys or a fishbone diagram to find why the defect occurs.
- **Preventive actions:** Coding standards, checklists, peer code reviews and static analysis.
- **Training:** Developers are trained on the common mistakes found.
- **Process update:** Update test cases and templates so the cause is closed permanently.
- **Result:** Fewer defects are introduced, so cost and rework drop and quality improves release by release.
- **Defect prevention:** It costs less than detection and correction.
- **Culture:** Lessons learned meeting after each release.

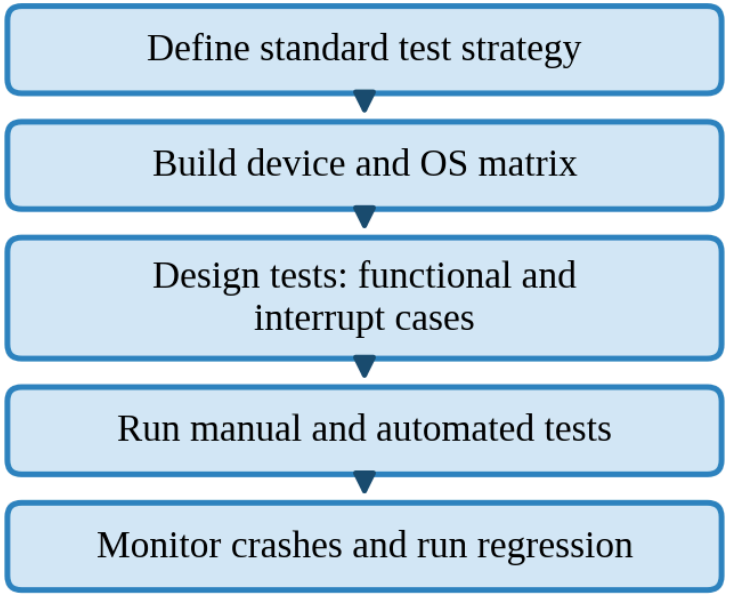


Question 16

A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.

Answer

- **Problem:** Testers follow different approaches, so crash scenarios get missed.
- **Standard strategy:** One documented strategy for levels, techniques, tools and exit criteria.
- **Principles applied:** Early testing, risk-based testing, defect clustering and context-dependent testing.
- **Device and OS matrix:** Most used devices, screen sizes and Android / iOS versions from analytics.
- **Mobile scenarios:** Low memory, network loss, incoming call, rotation and app switching.
- **Tools and monitoring:** Appium for automation, and crash reports feed new test cases.
- **Example:** App crashes when the network drops during payment, so a network-interruption test is added to every release.
- **Reliability:** Consistent coverage, repeatable results and fewer crashes in production.

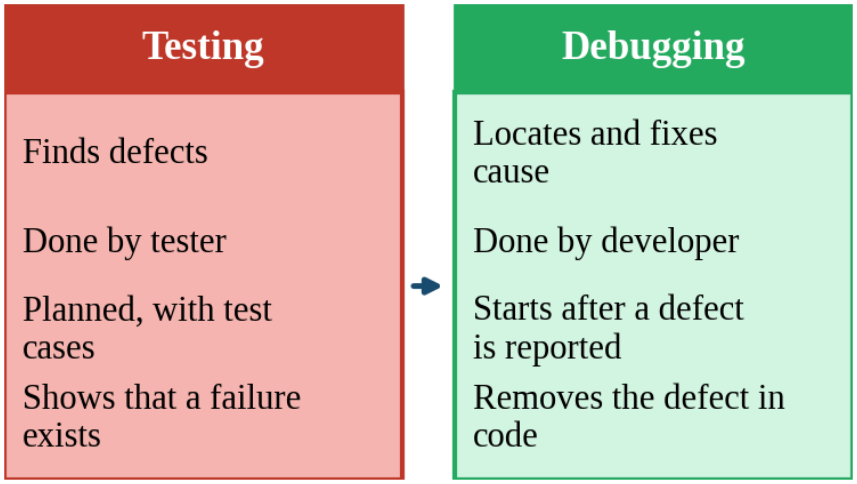


Question 17

A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer

- **Scenario:** A food-ordering app shows the wrong total after applying a coupon.
- **Testing:** The tester runs the coupon case, sees actual total differs from expected, and reports the defect.
- **Debugging:** The developer traces the code, finds that the discount is applied twice, and fixes it.
- **Then:** The tester retests and runs regression tests to confirm.
- **Roles:** Testing finds defects and debugging removes them. Both are needed for quality.
- **Key point:** Retesting confirms the fix. Regression testing confirms nothing else broke.
- **Difference:** Testing can be done without code knowledge. Debugging needs it.



Question 18

An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.

Answer

- **Choose browsers:** Use user analytics to pick browsers, versions and devices (Chrome, Edge, Firefox, Safari).
- **Compatibility matrix:** Test key flows (login, search, cart, payment) on each browser.
- **Risk-based:** Most-used browsers and critical flows first, since exhaustive testing is impossible.
- **Tools:** Selenium or BrowserStack for automated runs, with screenshots for defect reports.
- **Handling:** Fix browser-specific CSS/JS, then regression test on all browsers.
- **Responsive check:** Test different screen sizes and mobile browsers too.
- **Principle:** Testing is context dependent, so e-commerce needs payment and layout focus.

	Layout	Login	Cart	Payment
Chrome	Pass	Pass	Pass	Pass
Edge	Pass	Pass	Pass	Pass
Firefox	Pass	Pass	Pass	Pass
Safari	Pass	Pass	Pass	FAIL

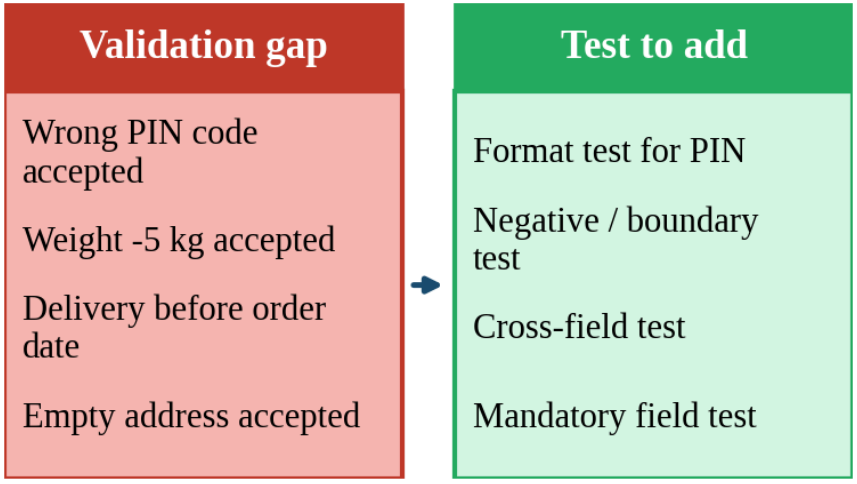
Compatibility matrix: Safari payment page needs a fix

Question 19

A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer

- **Problem:** Invalid or inconsistent data is accepted, so shipment, route and delivery outputs differ from what is expected.
- **Gap analysis:** Compare the validation rules in the requirements with the test cases, and list every rule that has no test.
- **Field-level gaps:** Data type, length, range and format of fields such as PIN code, weight and phone number.
- **Cross-field gaps:** Delivery date before order date, or source and destination being the same.
- **Techniques:** Equivalence partitioning, boundary value analysis and negative testing for each input.
- **Example:** A shipment weight of -5 kg or 0 kg is accepted and produces a wrong freight charge. A boundary and negative test would catch it.
- **Early detection:** Requirement and design reviews check that every validation rule is clear and testable.
- **Reliability:** Only valid data enters the system, so outputs are consistent, with fewer failures and less rework.

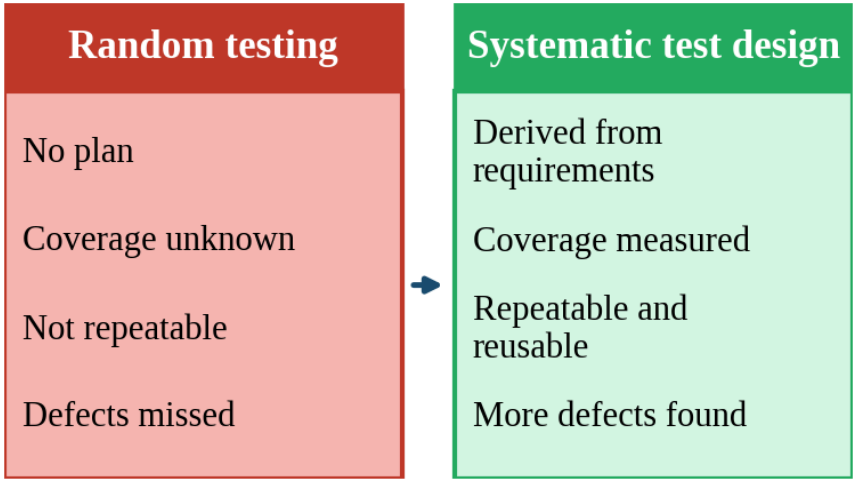


Question 20

A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer

- **Problem:** Random tests have no plan, so coverage is unknown, results cannot be repeated and defects are missed.
- **Engineering activity:** Testing is treated like any engineering task with analysis, design, implementation, execution and evaluation.
- **Systematic design:** Test conditions are derived from requirements using black-box techniques such as equivalence partitioning, BVA and decision tables.
- **Test case content:** ID, objective, preconditions, input, steps, expected result and actual result.
- **Traceability:** Each test case is linked to a requirement, so gaps in coverage are visible.
- **Example:** For a discount rule, a decision table gives every combination of customer type and amount, and random testing would try only a few.
- **Importance:** Better defect detection, repeatable results, reusable tests for regression and measurable progress.
- **Result:** Higher quality at lower cost.

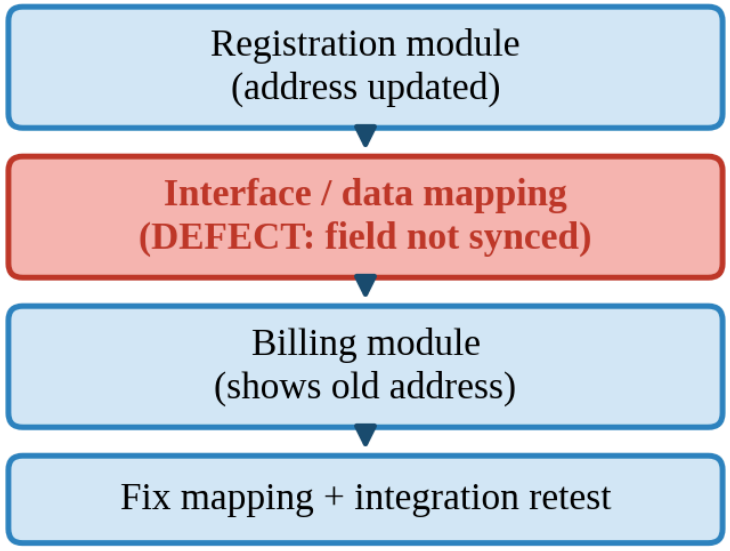


Question 21

A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer

- **Example:** A patient address updated in Registration still shows the old value in Billing.
- **Root causes:** Field mapping errors, different data formats, missing synchronisation or interface defects.
- **Principles used:** Early testing of interfaces, defect clustering at integration points and testing in context (patient data is critical).
- **Proper testing:** Integration tests compare data across modules, with data validation and end-to-end scenarios.
- **Result:** Same patient data everywhere, which is essential for safety.
- **Checks:** Compare record counts, field values and timestamps between modules.
- **Prevention:** Common data dictionary and interface specification.

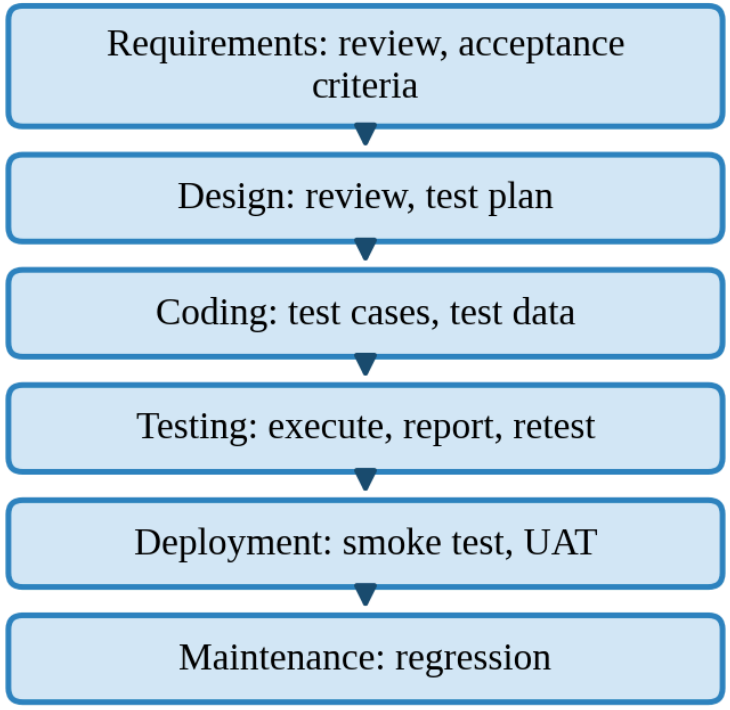


Question 22

A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer

- **Requirements:** Review requirements for clarity and testability, and define acceptance criteria.
- **Design:** Review design and interfaces, and prepare the test plan and test strategy.
- **Coding:** Design test cases, prepare test data and support code reviews and unit testing.
- **Testing:** Execute integration, system and regression tests, then report and retest defects.
- **Deployment:** Smoke testing and support for user acceptance testing.
- **Maintenance:** Regression testing after every change or patch.
- **Early involvement:** Defects are found when they are cheap to fix, and testers are ready to start the moment the build arrives, so there is no delay.
- **Collaboration:** Shared understanding between developers and testers means fewer reopened defects.

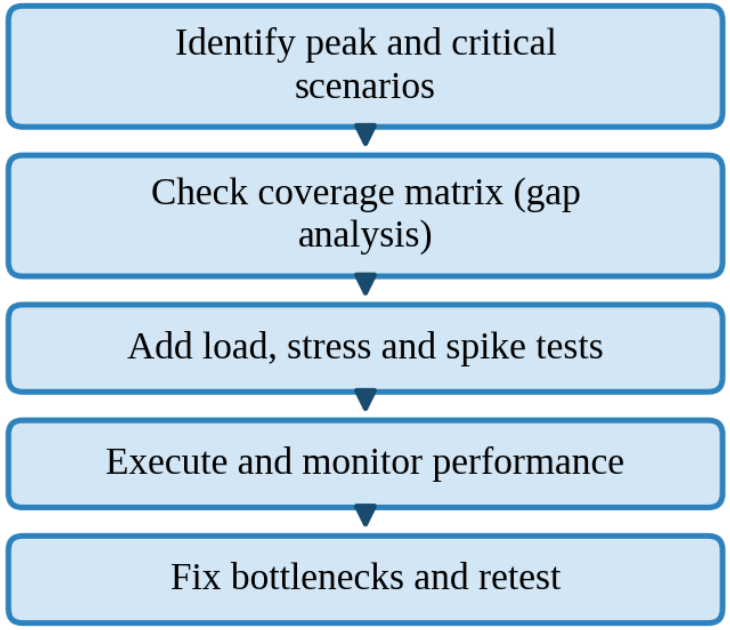


Question 23

A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer

- **Problem:** The system was tested with a few users, so heavy load and unusual combinations were never tried.
- **Principles applied:** Exhaustive testing is impossible, so risk-based testing picks the most critical scenarios, and testing is context dependent.
- **Gap identification:** Use a requirement traceability matrix and a coverage report to find requirements and scenarios with no test.
- **Missed scenarios:** Peak load, many users at once, large data volumes, sudden traffic spikes and long running sessions.
- **New tests:** Load, stress, spike and endurance tests with tools such as JMeter.
- **Example:** An online ticket site works for 100 users but fails when 10,000 book during a sale. A load test would find it before release.
- **Better coverage:** Requirement, scenario and risk coverage means every critical case has a test.
- **Result:** Failures are found in testing, not by customers, so the system stays stable at peak usage.

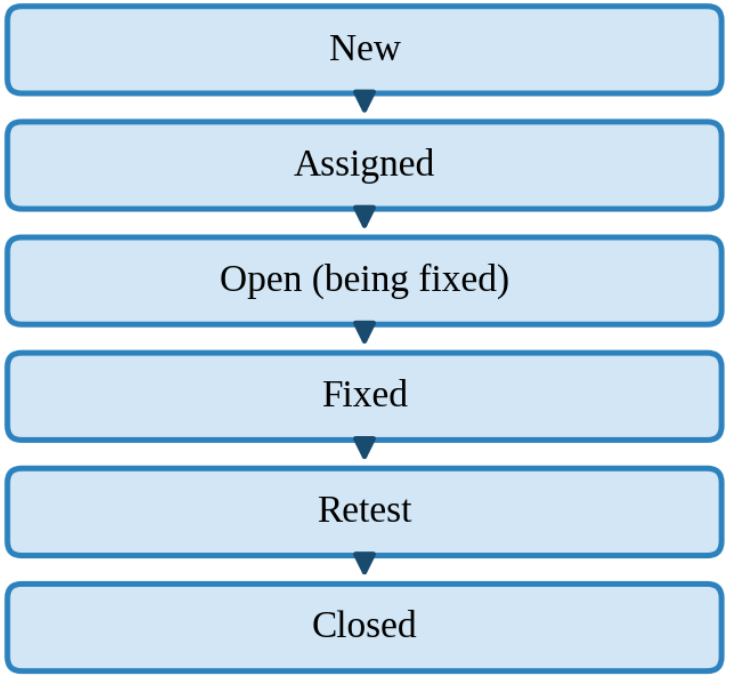


Question 24

A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer

- **Defect repository:** Use a tool such as Jira or Bugzilla as the single place for all defects.
- **Record:** ID, summary, steps to reproduce, expected vs actual result, severity, priority, environment and screenshots.
- **Track:** Status moves through the lifecycle with an owner at every stage.
- **Analyse:** Reports show open defects, defect density and defect trends by module.
- **Benefit:** Nothing is lost, developers can reproduce defects quickly and the data helps in prevention.
- **Severity vs priority:** Severity is technical impact. Priority is the order of fixing.
- **Reports:** Weekly defect summary for management decisions.

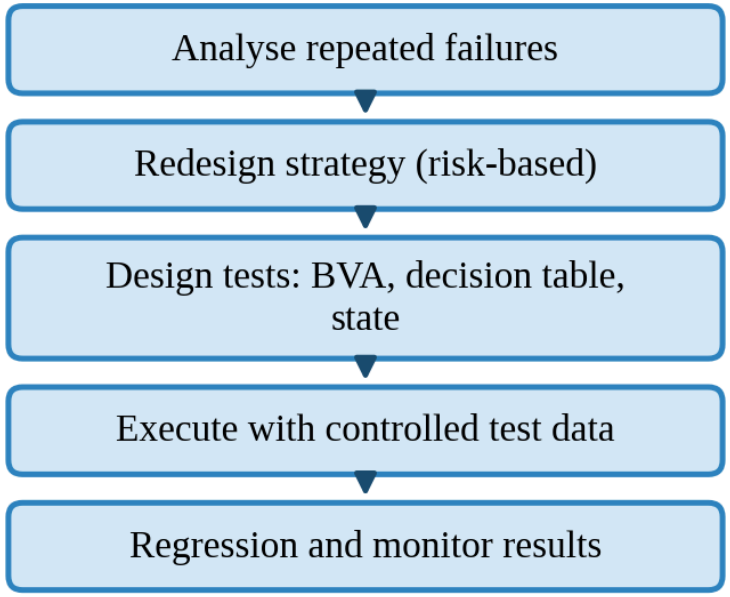


Question 25

A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer

- **Root cause:** Ad-hoc testing missed important transaction scenarios, so the same failures kept repeating.
- **Risk-based strategy:** Exhaustive testing is impossible, so critical flows (transfer, withdrawal, bill payment) are tested first and deepest.
- **Test design:** Boundary value analysis for amount and daily limits, decision tables for balance and limit rules, state transition tests for transaction states.
- **ACID checks:** Atomicity, consistency, isolation and durability are verified for every transaction.
- **Example:** In a Rs. 10,000 transfer the debit succeeds but the credit fails on timeout. A structured test checks that the amount is rolled back.
- **Environment and data:** A production-like test environment with controlled test data.
- **Regression:** Every fixed failure becomes a regression test and is automated.
- **Result:** Fewer failures, predictable releases, regulatory compliance and customer trust.

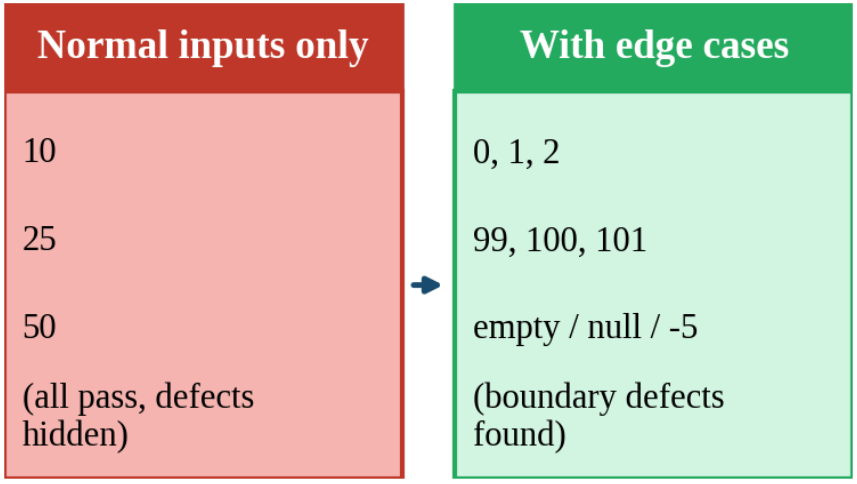


Question 26

A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Answer

- **Problem:** Normal values like 10, 25 or 50 pass, but the limits and unusual inputs were never tried.
- **Principle:** Exhaustive testing is impossible, so test cases are chosen where defects are most likely, which is at the edges.
- **Edge cases:** Minimum, maximum, just inside and just outside the limit, zero, negative, empty, null, and very long values.
- **Boundary value analysis:** Developers often make off-by-one mistakes such as < instead of <=, and BVA targets exactly those.
- **Example:** Quantity field accepts 1 to 100. Tests: 0, 1, 2, 99, 100, 101, empty and -5.
- **Defect detection:** A few boundary tests find defects that hundreds of normal tests miss.
- **Combine with:** Equivalence partitioning to keep the number of tests small.
- **Result:** Better coverage, earlier defect detection and more robust software.

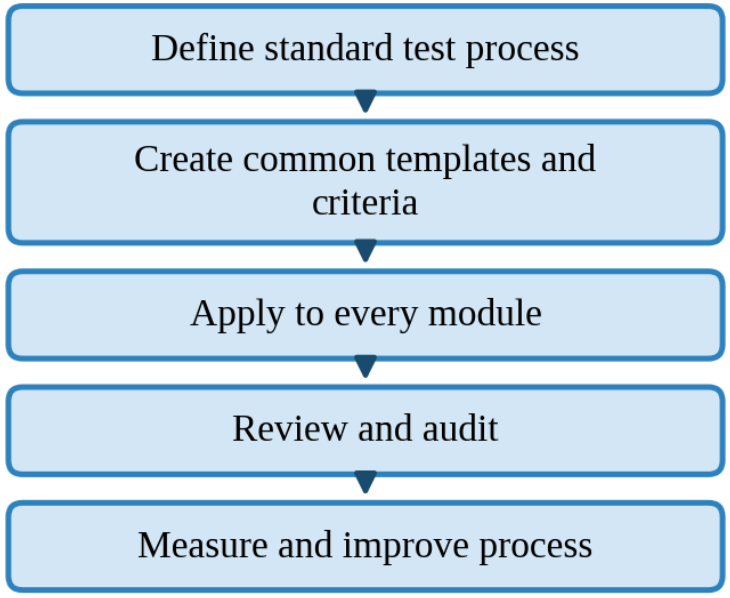


Question 27

A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer

- **Problem:** Each module is tested in its own way, so some modules get deep testing and others get very little.
- **Standard process:** One defined test process for all modules: planning, design, execution, reporting and closure.
- **Common templates:** Same formats for test plan, test case, defect report and test summary report.
- **Entry and exit criteria:** A module is released only when the same quality criteria are met.
- **Standard techniques and metrics:** BVA, equivalence partitioning, coverage and defect density are used and measured the same way everywhere.
- **Review and audit:** Test cases are peer reviewed and the process is audited.
- **Example:** Login, cart and payment modules all follow the same checklist, so quality is even.
- **Result:** Even quality, easy comparison of modules, less rework and easier training of new testers.

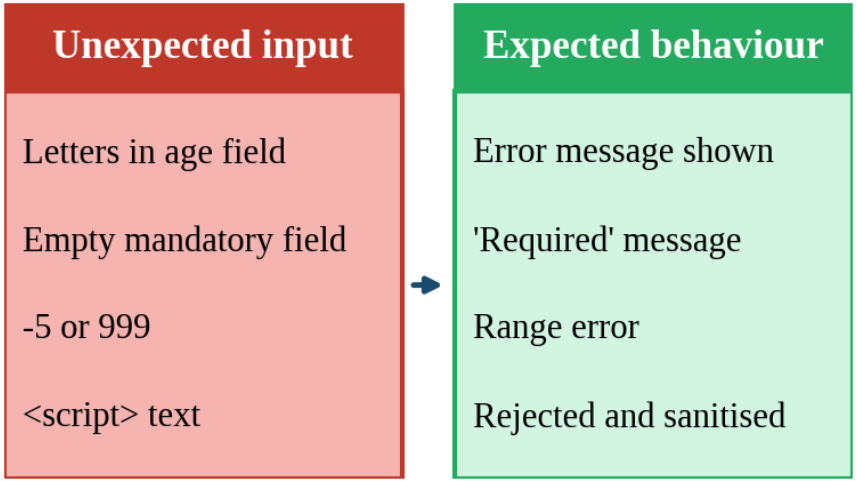


Question 28

A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

Answer

- **Problem:** The application assumed users always enter valid data, so unexpected inputs caused failures.
- **Validation checks:** Data type, length, range, format and mandatory fields are checked on both client and server side.
- **Negative testing:** Test cases are written for invalid inputs, not only valid ones.
- **Techniques:** Equivalence partitioning for valid and invalid classes and boundary value analysis at the limits.
- **Example:** In an age field, enter letters, blank, -5, 999 and special characters. The system must show a clear error and not crash.
- **Security link:** Special characters and script text are tested to prevent SQL injection and cross-site scripting.
- **Robustness:** The system handles bad input gracefully, with proper messages and without data corruption.
- **Result:** Reliable and secure application with a better user experience.

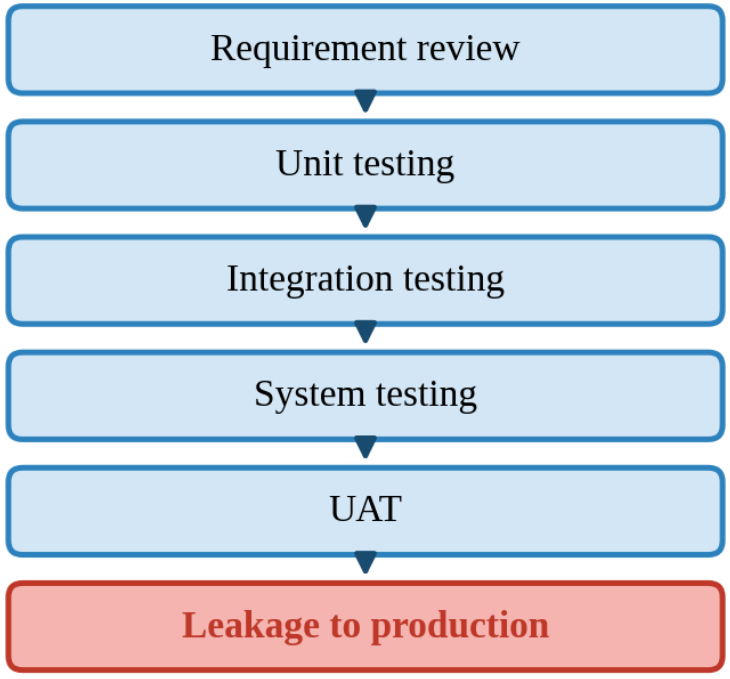


Question 29

A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer

- **Defect leakage:** Defects missed in testing and found by users. DLR = production defects / (testing + production defects) x 100.
- **Root causes:** Missed test scenarios, weak requirement review, poor regression, test environment unlike production and time pressure.
- **Principles:** Early testing, risk-based testing (exhaustive is impossible), defect clustering and pesticide paradox (update test cases).
- **Improvements:** Better coverage, updated regression suite, production-like environment and root cause analysis of every leaked defect.
- **Result:** Fewer defects reach customers.
- **Metric:** Track the leakage rate every release to measure improvement.
- **Feedback loop:** Production defects become new test cases.



Question 30

A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.

Answer

- **Principle:** Early testing. Testing activities start from the requirement phase, and they are planned in parallel with development.
- **Static testing:** Reviews, walkthroughs and inspections of requirements, design and code find defects without running the software.
- **Early test design:** Test cases are written while requirements are being finalised, which also exposes ambiguities.
- **Unit testing and continuous integration:** Developers test each build, so integration problems are found within hours.
- **Risk-based planning:** High-risk modules are tested first, using defect history.
- **Example:** A missing validation rule found in a design review is fixed in minutes. After release it needs a patch, retesting and customer support.
- **Efficiency:** Less rework, lower cost, shorter schedule and fewer defects leaking to production.
- **Result:** Better quality at lower overall effort.

